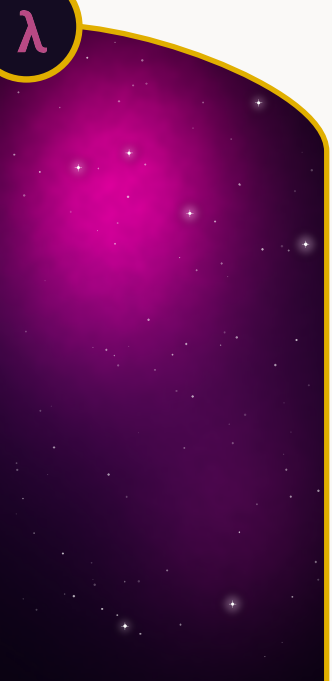


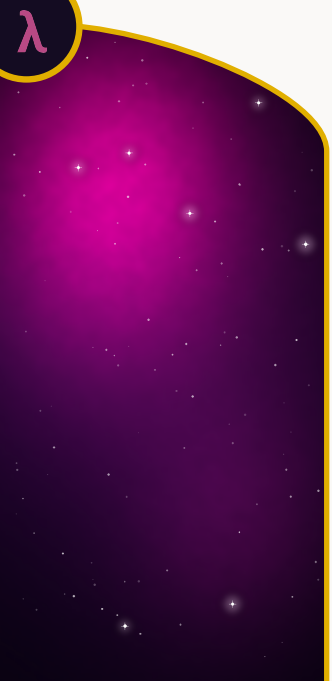
L2B: Orders of Growth

CS1101S: Programming Methodology

Boyd Anderson

21st August, 2026

Version 1.1 — Built: 2026-08-21 06:11:58



Outline

Announcements

Tree Recursion (Textbook 1.2.2)

Orders of Growth (Textbook 1.2.3)

Calendars & Absence Policy

New Calendars

Now on Canvas: **Lectures and Events** calendar and **Timelines** calendar — subscribe to stay in sync!

Calendars & Absence Policy

New Calendars

Now on Canvas: **Lectures and Events** calendar and **Timelines** calendar — subscribe to stay in sync!

Simplified Absence Policy

Allowed without penalty, throughout the semester:

- 1 absence in Studios

Calendars & Absence Policy

New Calendars

Now on Canvas: **Lectures and Events** calendar and **Timelines** calendar — subscribe to stay in sync!

Simplified Absence Policy

Allowed without penalty, throughout the semester:

- 1 absence in Studios
- 1 absence in Reflections

Calendars & Absence Policy

New Calendars

Now on Canvas: **Lectures and Events** calendar and **Timelines** calendar — subscribe to stay in sync!

Simplified Absence Policy

Allowed without penalty, throughout the semester:

- 1 absence in Studios
- 1 absence in Reflections
- 2 absences in Lectures

Calendars & Absence Policy

New Calendars

Now on Canvas: **Lectures and Events** calendar and **Timelines** calendar — subscribe to stay in sync!

Simplified Absence Policy

Allowed without penalty, throughout the semester:

- 1 absence in Studios
- 1 absence in Reflections
- 2 absences in Lectures
- 1 absence in RA/PA

E.g. for Studios: we expect **10 non-zero Studio XP** out of 11 Studios for full attendance credit.

Calendars & Absence Policy

New Calendars

Now on Canvas: **Lectures and Events** calendar and **Timelines** calendar — subscribe to stay in sync!

Simplified Absence Policy

Allowed without penalty, throughout the semester:

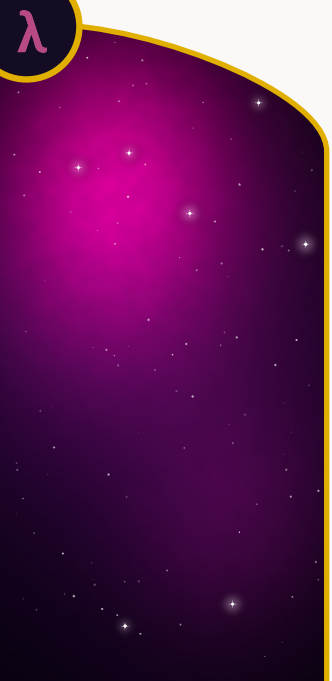
- 1 absence in Studios
- 1 absence in Reflections
- 2 absences in Lectures
- 1 absence in RA/PA

E.g. for Studios: we expect **10 non-zero Studio XP** out of 11 Studios for full attendance credit.

Keep your MCs — only submit them at the end of semester if you exceed the allowance above.

Studio Attendance and Participation

- Attendance and Participation (5% of CS1101S)
  - Studios are small communities of learners; your presence in the sessions is needed to be an effective member
  - 3% for Studio attendance
  - 2% for Studio participation; Avengers assess your participation, based on effort
  - The ideal Studio member prepares well for the meetings, contributes by answering any questions thoughtfully, asks good questions, and tries to help classmates in mastering the material
- Studio Performance XP
  - Maximal 500 XP per Studio session
  - Here is where you can shine, by helping others, contributing to discussions, going the extra mile

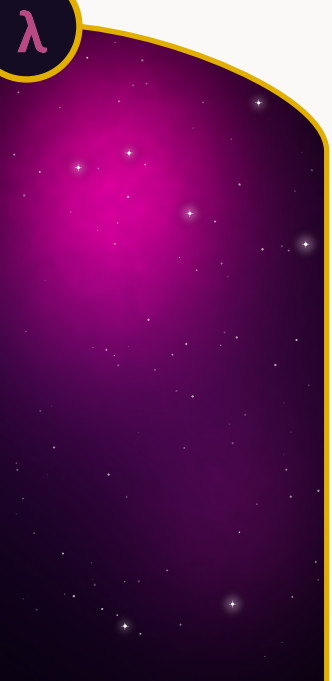


Outline

Announcements

Tree Recursion (Textbook 1.2.2)

Orders of Growth (Textbook 1.2.3)



Fibonacci Numbers

- Fibonacci sequence
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
  - Each number is the sum of the previous two

Fibonacci Numbers

- Fibonacci sequence
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
  - Each number is the sum of the previous two
- Fibonacci function F(n), such that $F(0) = 0$, $F(1) = 1$, $F(2) = 1$, $F(3) = 2$, $F(4) = 3$, and so on

Fibonacci Numbers

- Fibonacci sequence
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
  - Each number is the sum of the previous two
- Fibonacci function F(n), such that $F(0) = 0$, $F(1) = 1$, $F(2) = 1$, $F(3) = 2$, $F(4) = 3$, and so on
- Our attempt to implement F(n) in Python:

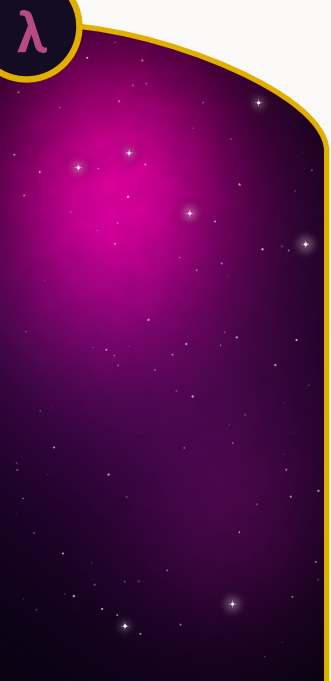
Fibonacci Numbers

- Fibonacci sequence
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
  - Each number is the sum of the previous two
- Fibonacci function F(n), such that $F(0) = 0$, $F(1) = 1$, $F(2) = 1$, $F(3) = 2$, $F(4) = 3$, and so on
- Our attempt to implement F(n) in Python:

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```

Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



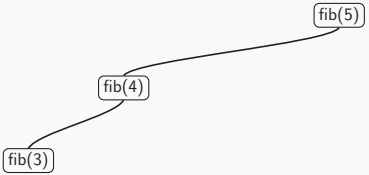
fib(5)

Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```

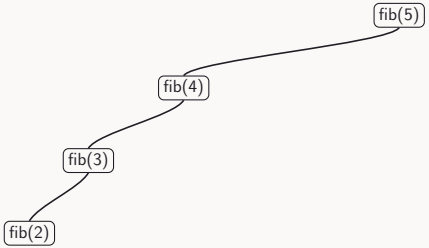
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



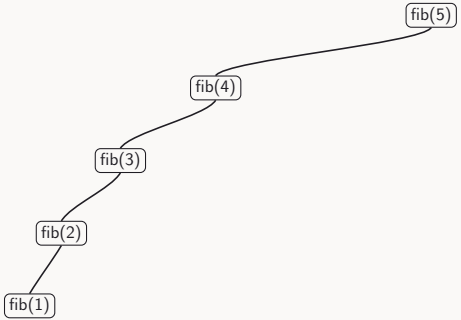
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



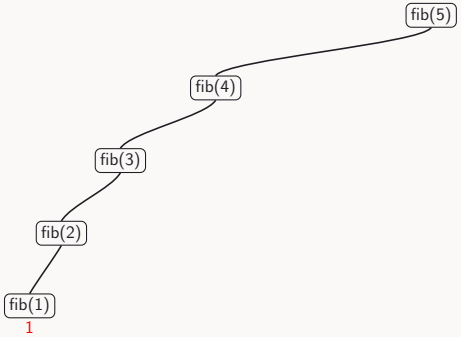
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



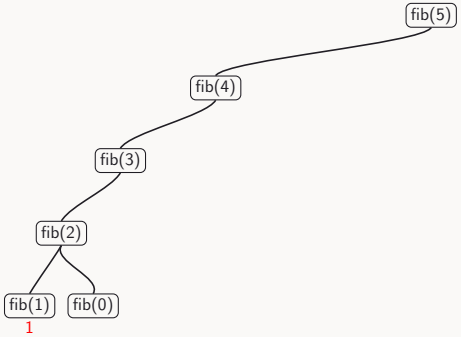
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



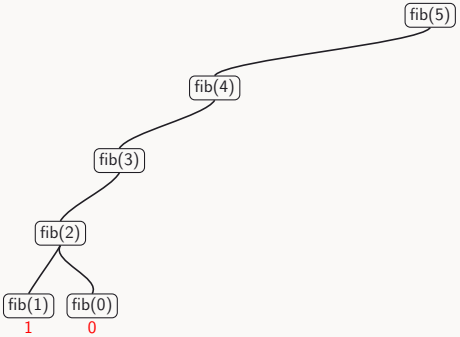
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



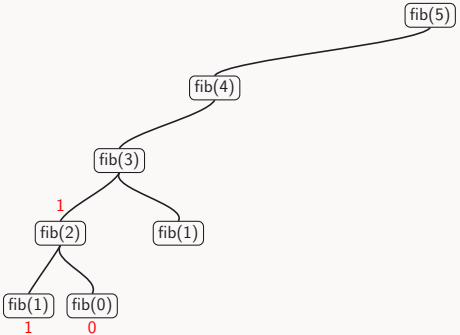
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



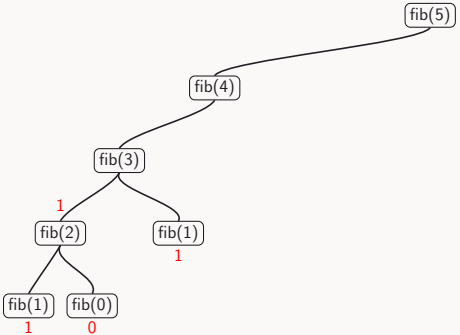
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



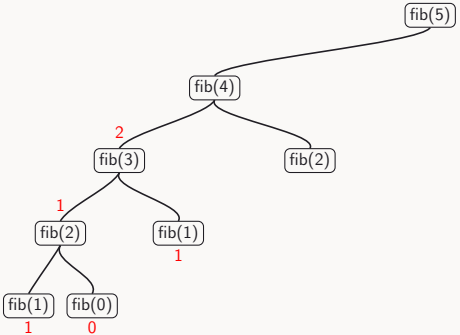
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



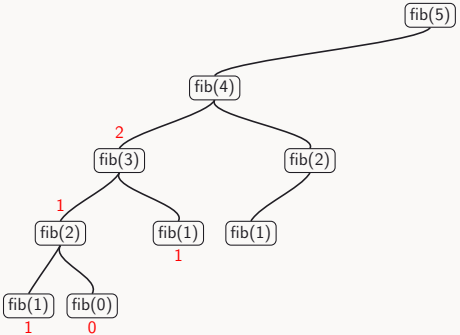
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



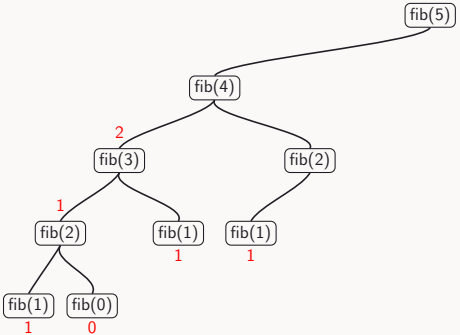
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



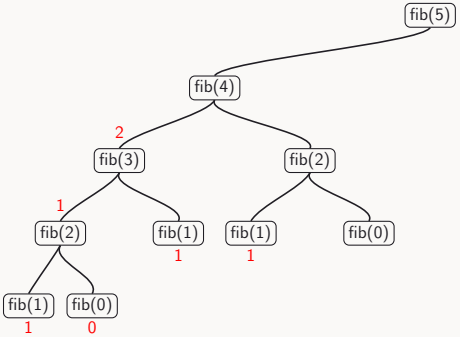
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



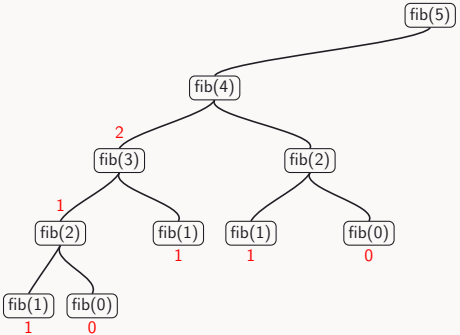
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



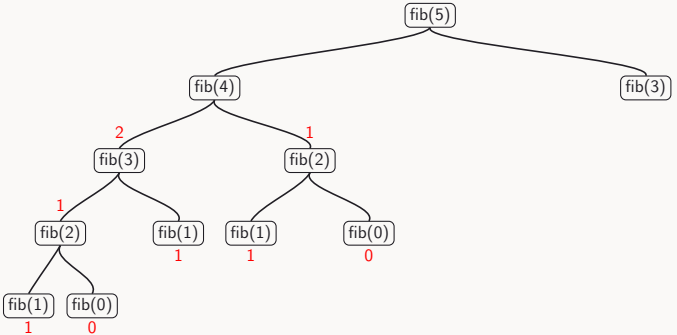
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



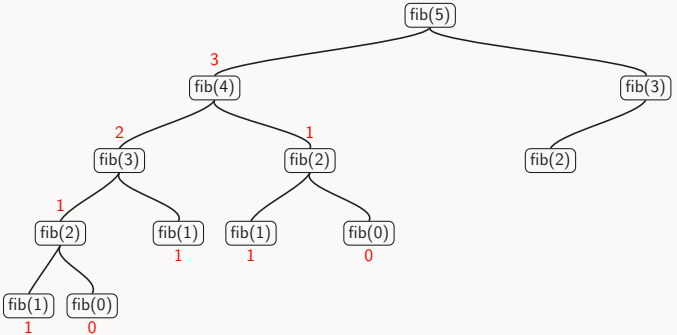
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



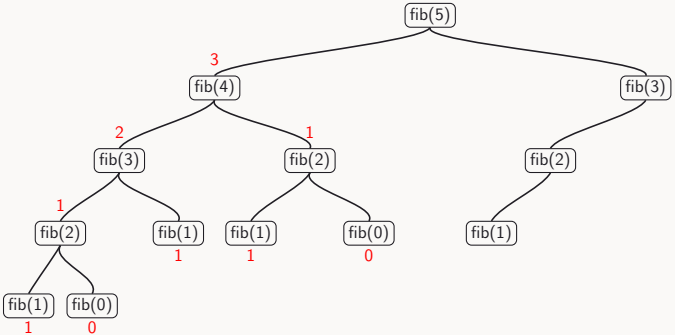
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



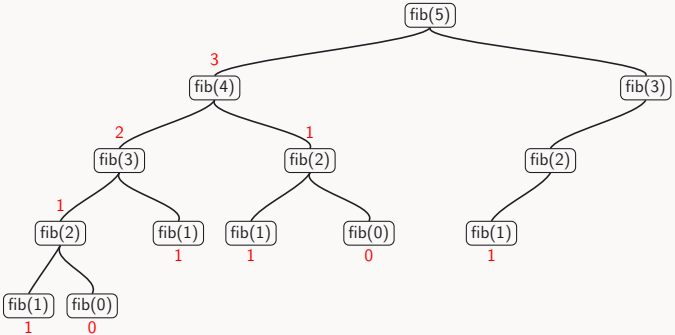
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



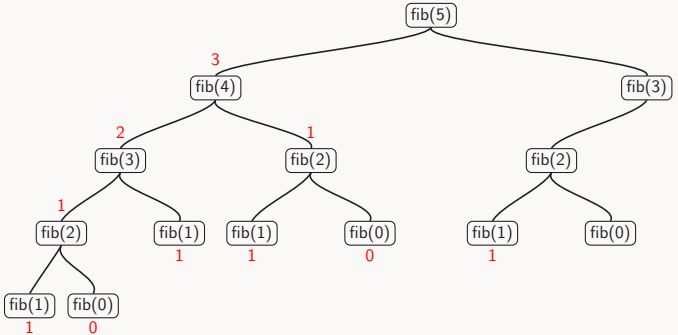
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



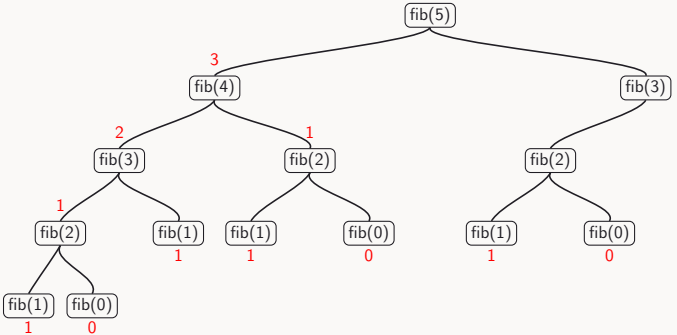
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



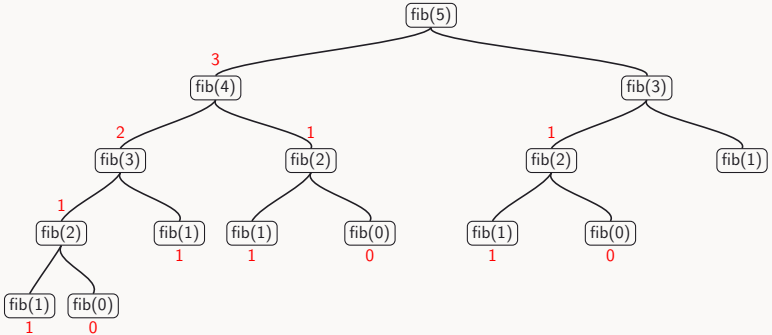
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



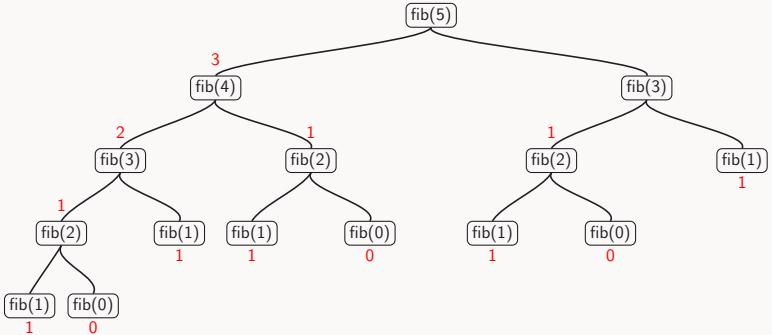
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



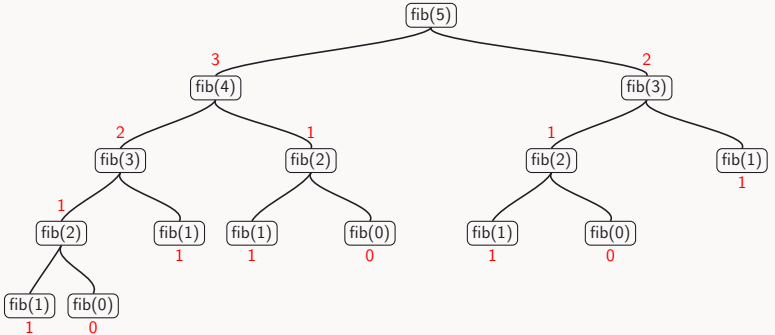
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



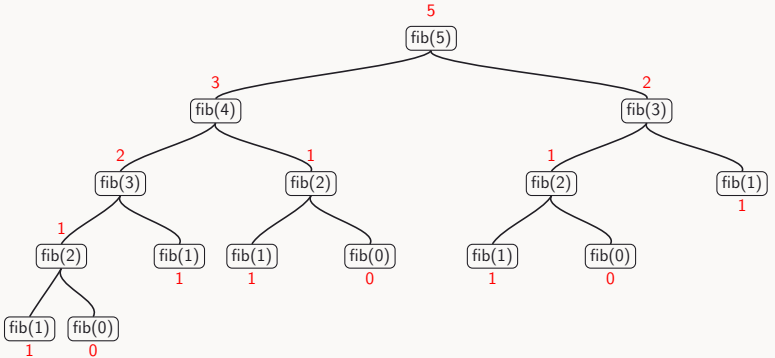
Evaluating fib(5)

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```

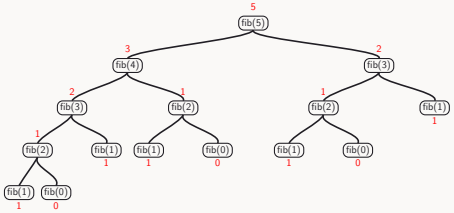


Evaluating fib(5)

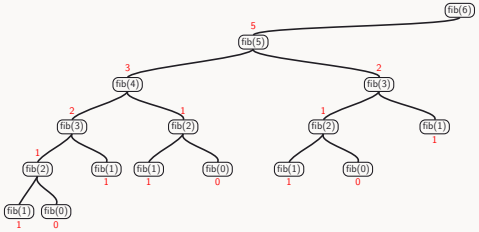
```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```



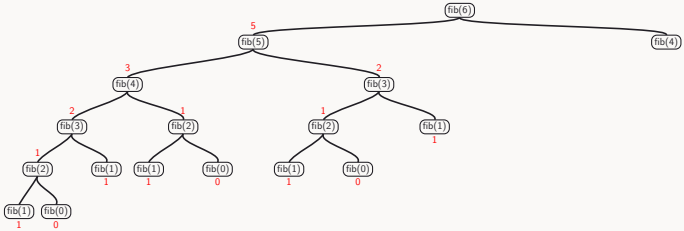
Evaluating fib(6)



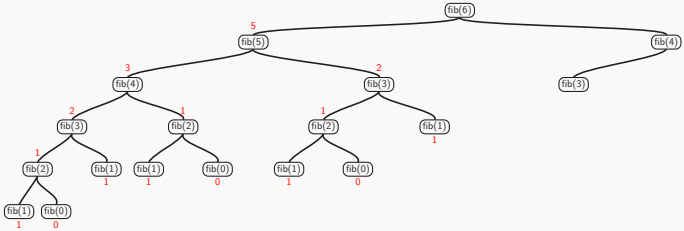
Evaluating fib(6)



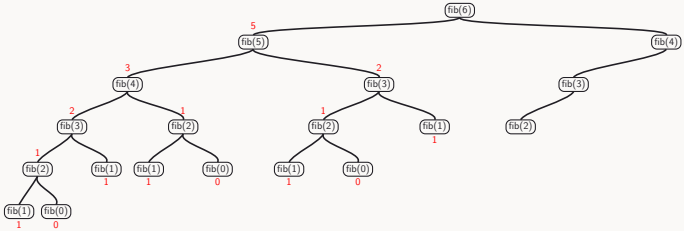
Evaluating fib(6)



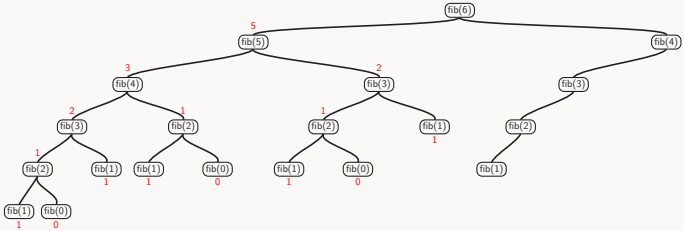
Evaluating fib(6)



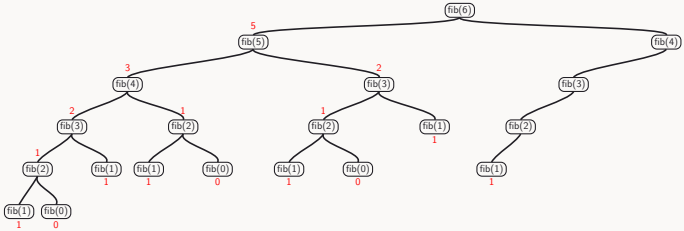
Evaluating fib(6)



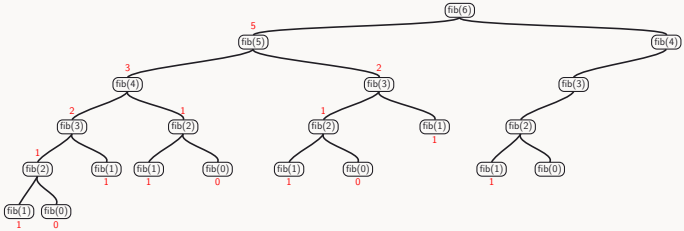
Evaluating fib(6)



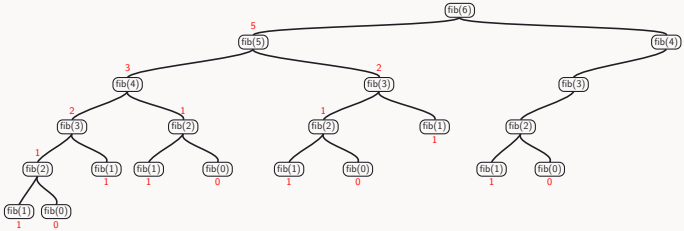
Evaluating fib(6)



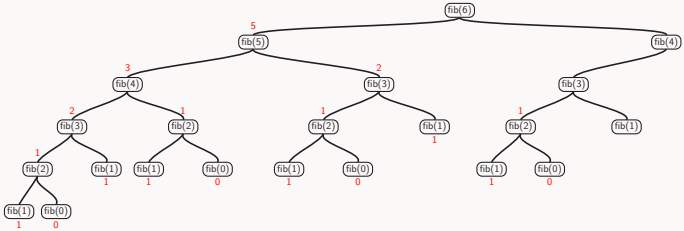
Evaluating fib(6)



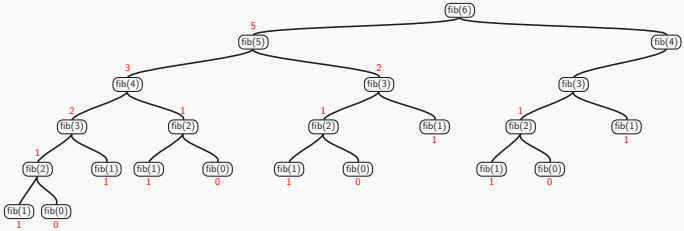
Evaluating fib(6)



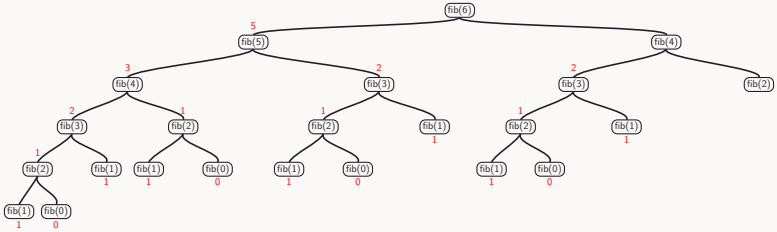
Evaluating fib(6)



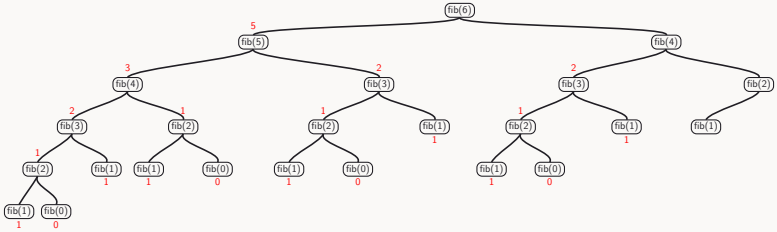
Evaluating fib(6)



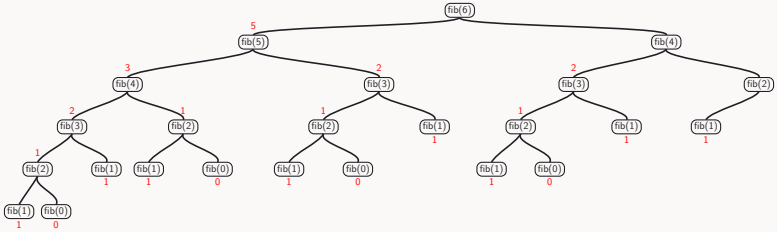
Evaluating fib(6)



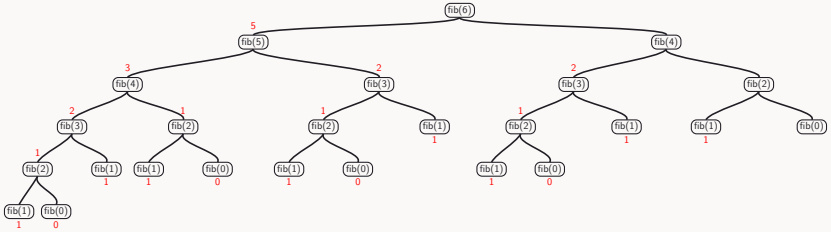
Evaluating fib(6)



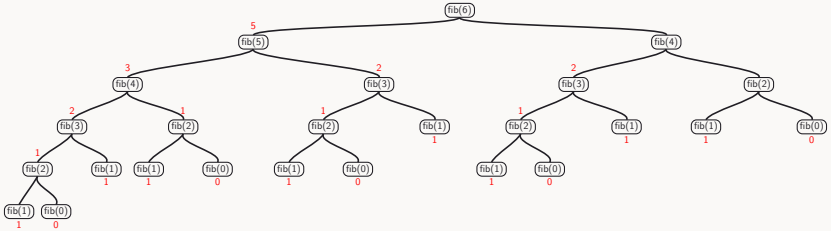
Evaluating fib(6)



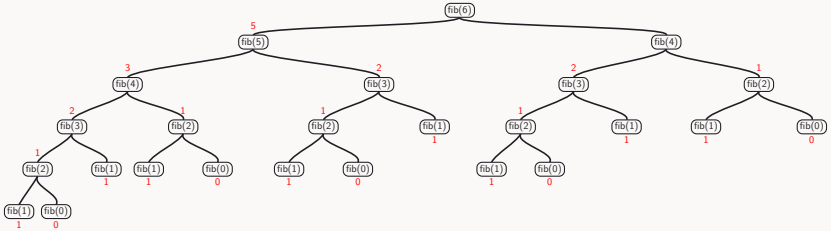
Evaluating fib(6)



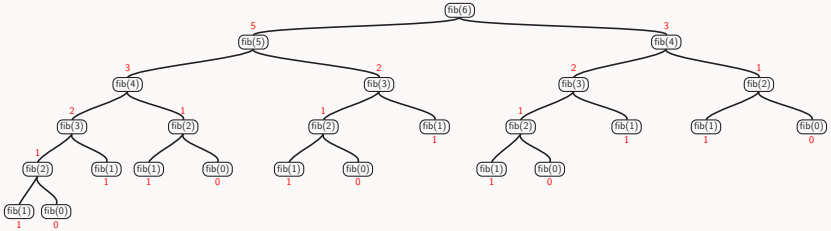
Evaluating fib(6)



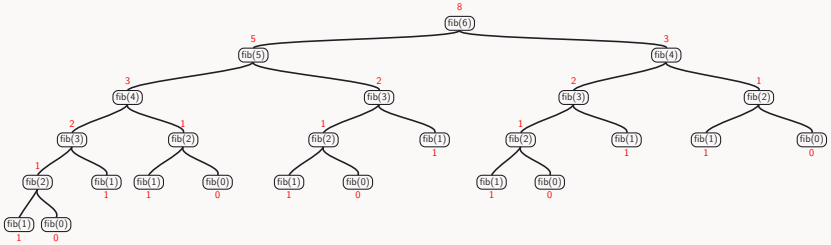
Evaluating fib(6)



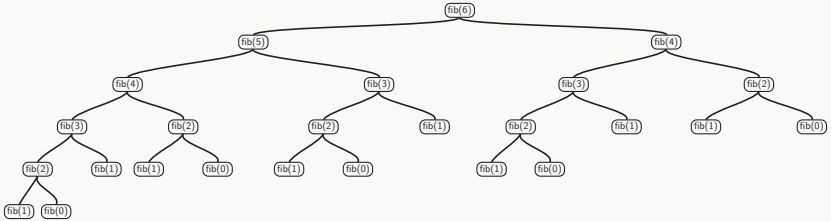
Evaluating fib(6)



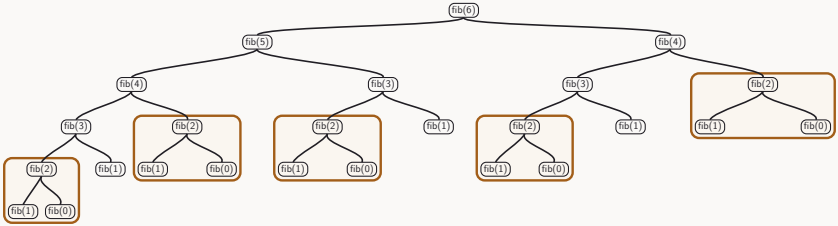
Evaluating fib(6)



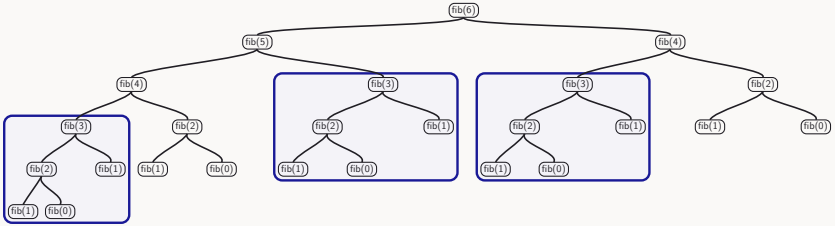
Evaluating fib(6)



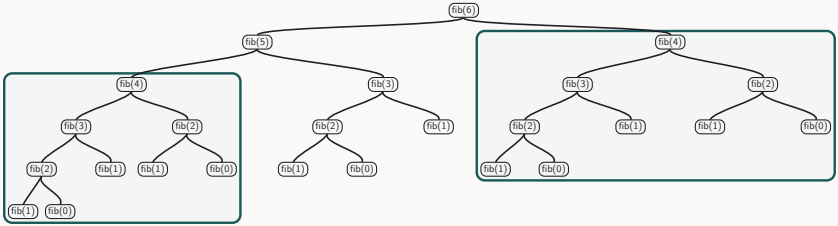
Evaluating fib(6)



Evaluating fib(6)



Evaluating fib(6)

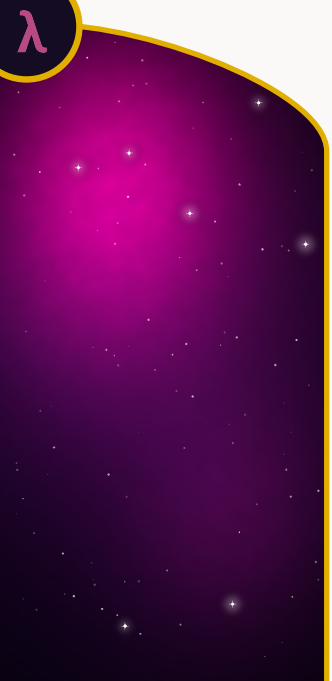


Example: Computing F(n) using fib(n)

- Number of leaves in the recursion tree for fib(n) is F(n + 1)
  - fib(10) needs to visit $F(11) = 89$ leaves
  - fib(20) needs to visit $F(21) = 10946$ leaves
  - fib(40) needs to visit $F(41) = 165580141$ leaves
  - fib(100) needs to visit $F(101) = 573147844013817084101$ leaves

Time for Evaluating fib(n)

- Time for exploring the tree grows with size of tree



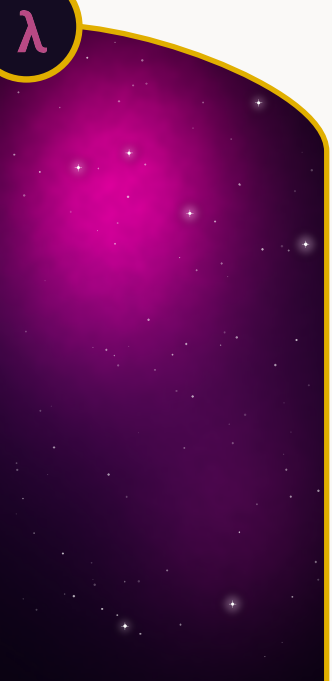
Time for Evaluating fib(n)

- Time for exploring the tree grows with size of tree
- Tree for fib(n) has F(n + 1) leaves, where
  - $F(n) = \left\lfloor \frac{\varphi^n}{\sqrt{5}} + \frac{1}{2} \right\rfloor$, and $\varphi = \frac{1+\sqrt{5}}{2}$

Time for Evaluating fib(n)

- Time for exploring the tree grows with size of tree
- Tree for fib(n) has F(n + 1) leaves, where
  - $F(n) = \left\lfloor \frac{\varphi^n}{\sqrt{5}} + \frac{1}{2} \right\rfloor$, and $\varphi = \frac{1+\sqrt{5}}{2}$

Can we write an efficient iterative function that computes F(n)?



Fibonacci Numbers (iterative version)

Fibonacci Numbers (iterative version)

```
def fib(n):
    return fib_iter(1, 0, n)

def fib_iter(a, b, count):
    return b if count == 0 else fib_iter(a + b, a, count
    - 1)
```

Fibonacci Numbers (iterative version)

```
def fib(n):
    return fib_iter(1, 0, n)

def fib_iter(a, b, count):
    return b if count == 0 else fib_iter(a + b, a, count
    - 1)
```

- How efficient is this program?

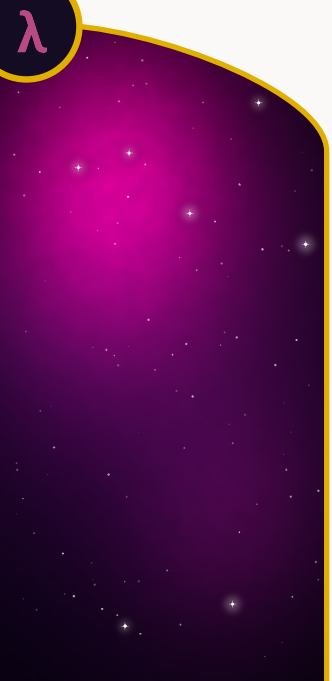
Fibonacci Numbers (iterative version)

```
def fib(n):
    return fib_iter(1, 0, n)

def fib_iter(a, b, count):
    return b if count == 0 else fib_iter(a + b, a, count
    - 1)
```

- How efficient is this program?

Wait. What does it even mean to be efficient?



Outline

Announcements

Tree Recursion (Textbook 1.2.2)

Orders of Growth (Textbook 1.2.3)

A Tale of Two Programmers

Prof Martin
writes beautiful, elegant code

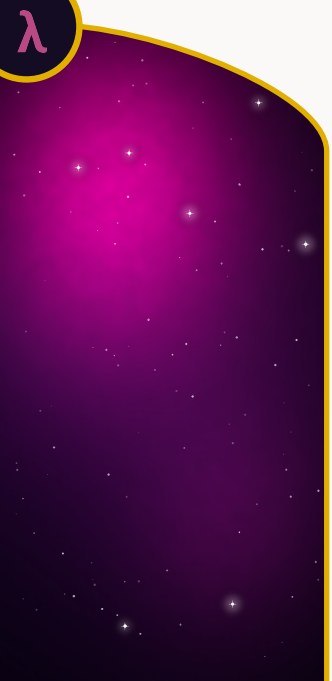
Prof Sanka
writes ruthlessly efficient code

A Tale of Two Programmers

Prof Martin
writes beautiful, elegant code

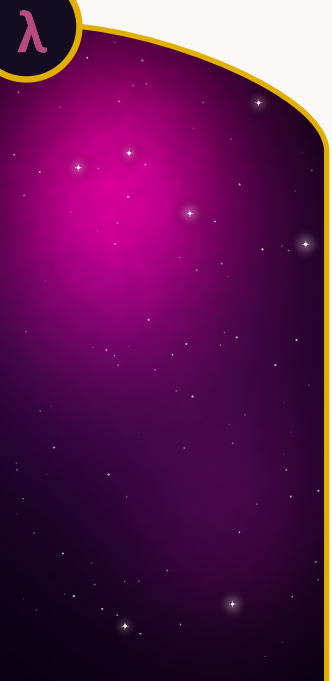
Prof Sanka
writes ruthlessly efficient code

Both are asked to solve the *same* problem.



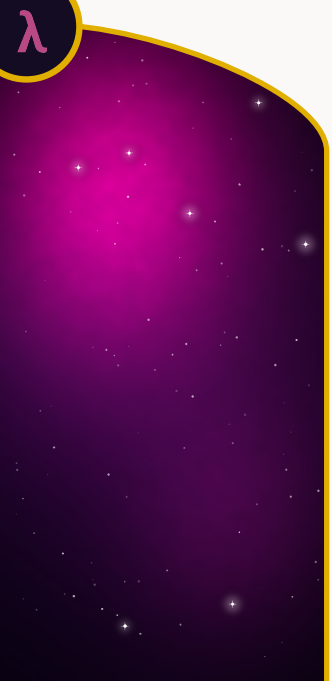
How Do We Compare Their Programs?

- Readability



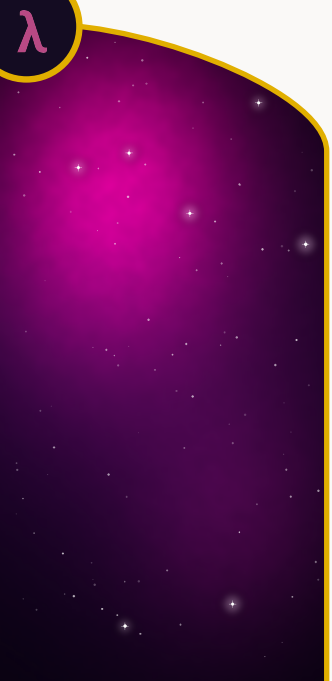
How Do We Compare Their Programs?

- Readability
- Style



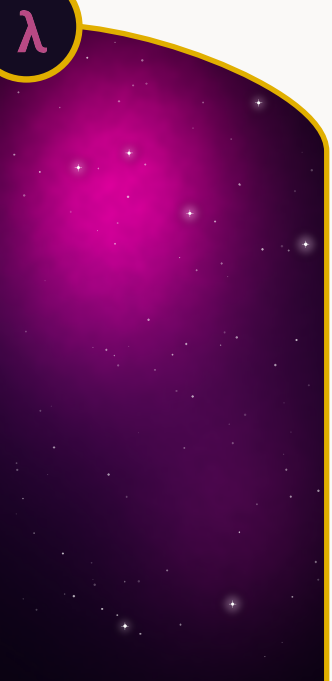
How Do We Compare Their Programs?

- Readability
- Style
- Elegance



How Do We Compare Their Programs?

- Readability
- Style
- Elegance
- ...



How Do We Compare Their Programs?

- Readability
- Style
- Elegance
- ...

Wait, how do we compare programs?

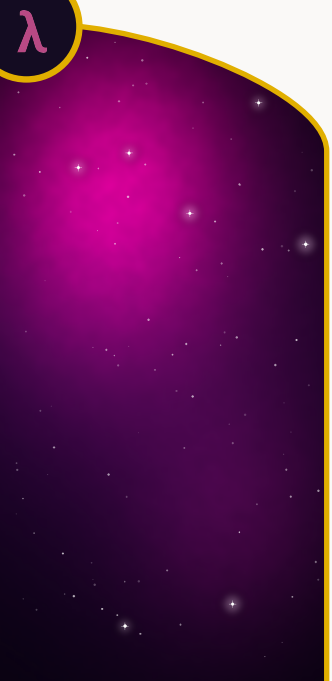
How Do We Compare Their Programs?

- Readability
- Style
- Elegance
- ...

Wait, how do we compare programs?

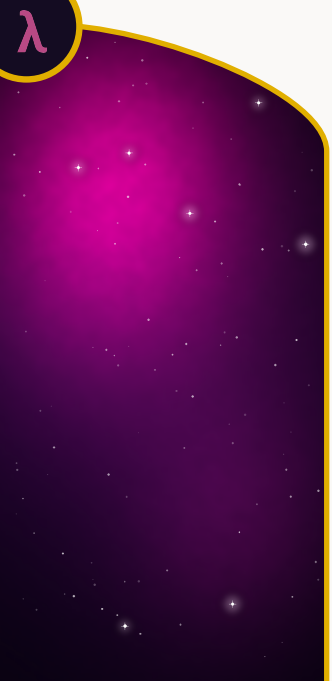
Today: two measurable dimensions

Time: how long does the program run
Space: how much memory does it need



Recall the Factorial Function

- Recursive definition
  - $n! = 1$ if $n = 1$
  - $= n(n-1)!$ if $n > 1$



Recall the Factorial Function

- Recursive definition
  - $n! = 1$ if $n = 1$
  - $= n(n-1)!$ if $n > 1$
- In Python:

Recall the Factorial Function

- Recursive definition
  - $n! = 1$ if $n = 1$
  - $= n(n-1)!$ if $n > 1$
- In Python:

```
def factorial(n):
    return 1 if n == 1 else n * factorial(n - 1)
```

Recall the Factorial Function

- Recursive definition
  - $n! = 1$ if $n = 1$
  - $= n(n-1)!$ if $n > 1$
- In Python:

```
def factorial(n):
    return 1 if n == 1 else n * factorial(n - 1)
```

- Example call:

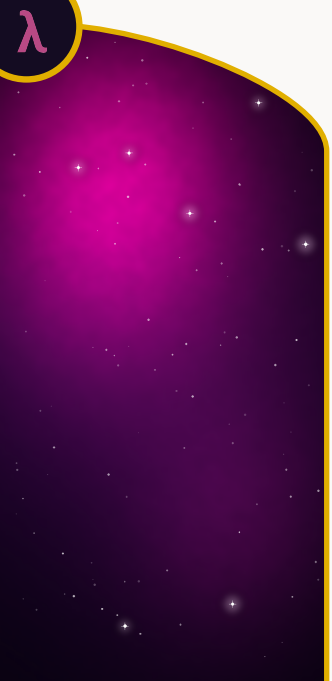
Recall the Factorial Function

- Recursive definition
  - $n! = 1$ if $n = 1$
  - $= n(n-1)!$ if $n > 1$
- In Python:

```
def factorial(n):
    return 1 if n == 1 else n * factorial(n - 1)
```

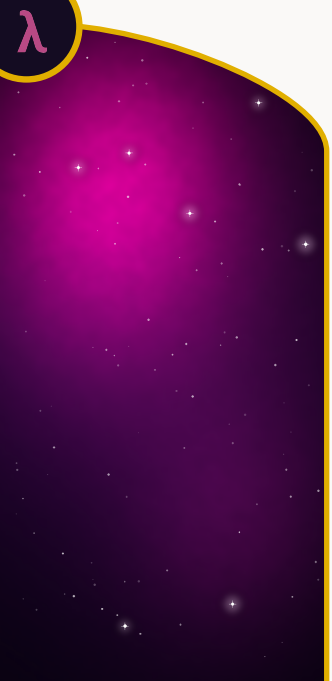
- Example call:

```
print(factorial(4))
```



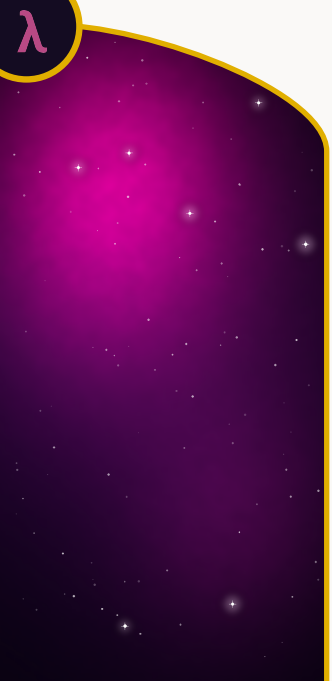
Time for Calculating factorial(n)

```
factorial(4)
```



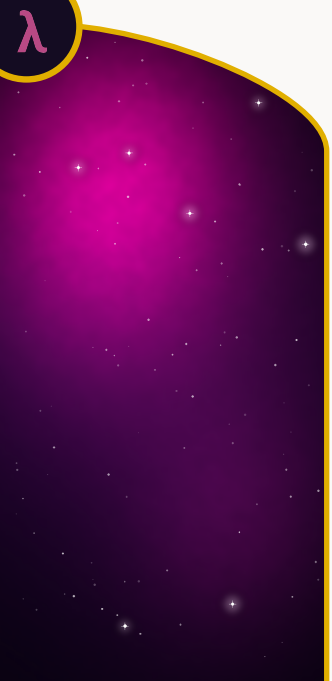
Time for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
```



Time for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
```



Time for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
```

Time for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
```

Time for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
```

Time for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
```

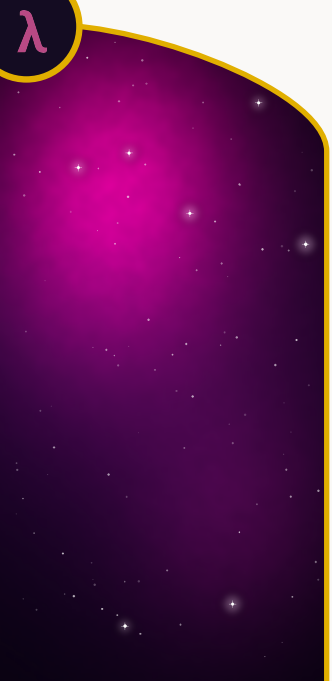
Time for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
24
```

Time for Calculating factorial(n): Observation

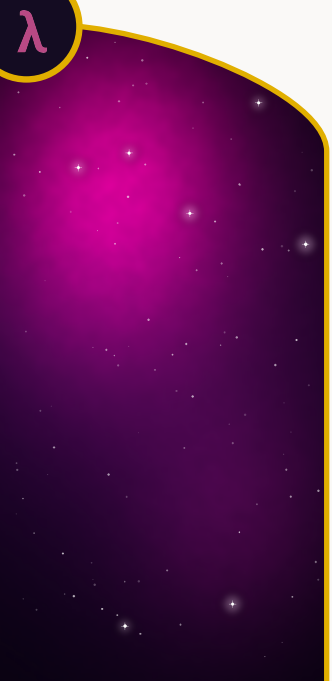
Observation

Number of operations grows linearly proportional to n



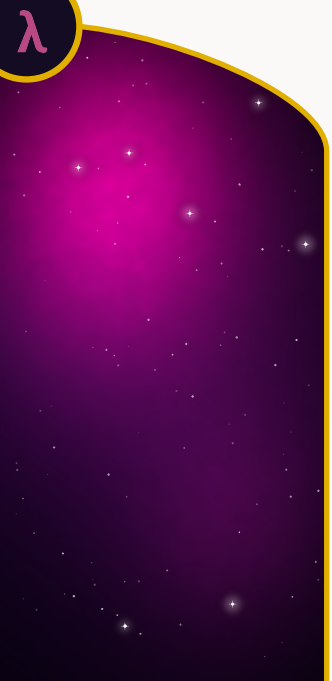
Space for Calculating factorial(n)

```
factorial(4)
```



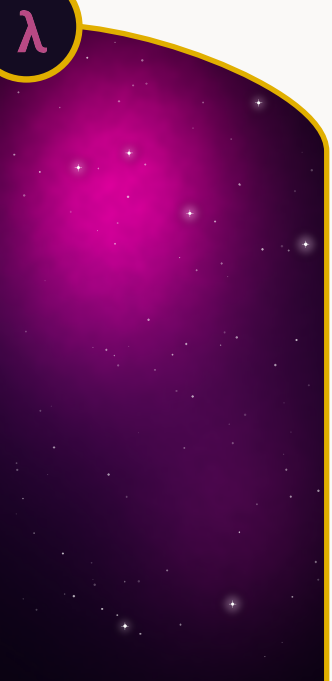
Space for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
```



Space for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
```



Space for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
```

Space for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
```

Space for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
```

Space for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
```

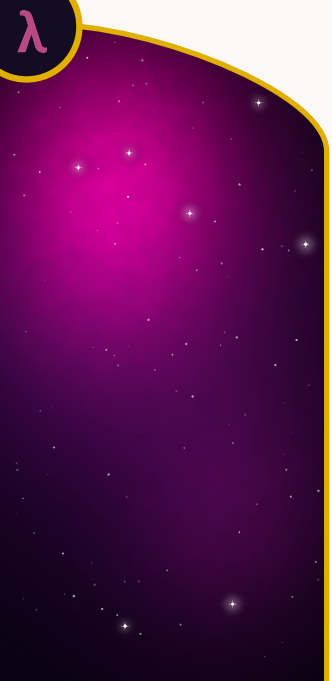
Space for Calculating factorial(n)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
24
```

Space for Calculating factorial(n): Observation

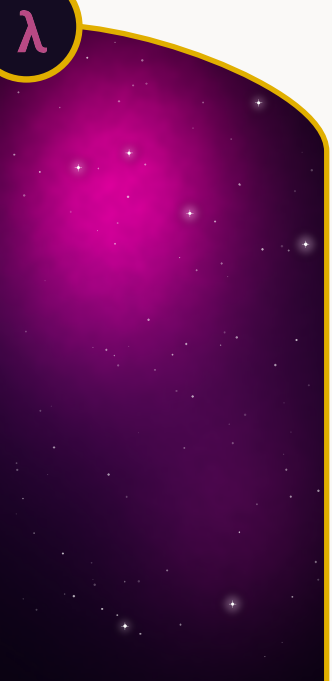
Observation

Number of deferred operations grows linearly proportional to n



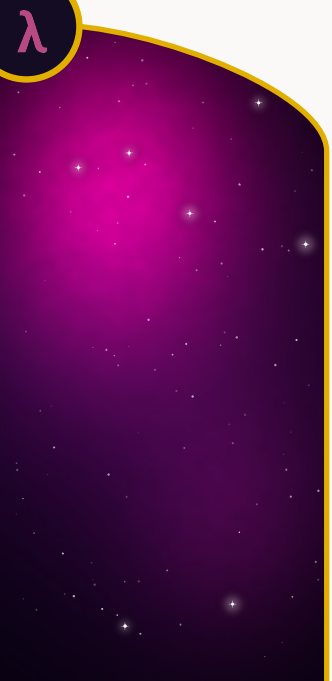
Doubling the Argument of factorial (n = 2)

```
factorial(2)
```



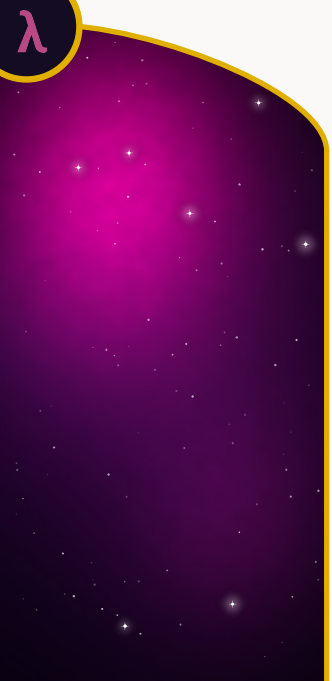
Doubling the Argument of factorial (n = 2)

```
factorial(2)
2 * factorial(1)
```



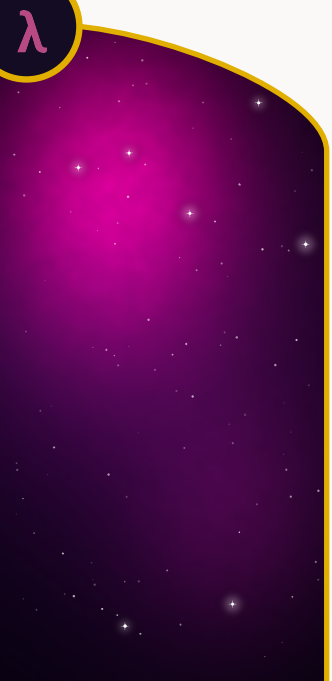
Doubling the Argument of factorial (n = 2)

```
factorial(2)
2 * factorial(1)
2 * 1
```



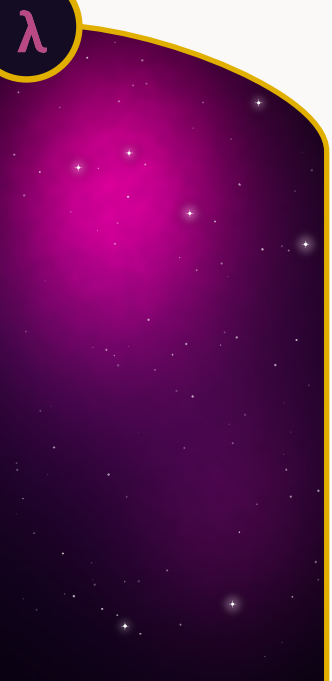
Doubling the Argument of factorial (n = 2)

```
factorial(2)
2 * factorial(1)
2 * 1
2
```



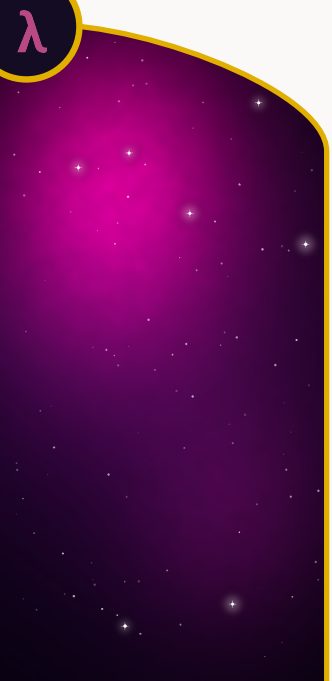
Doubling the Argument of factorial (n = 4)

```
factorial(4)
```



Doubling the Argument of factorial (n = 4)

```
factorial(4)
4 * factorial(3)
```



Doubling the Argument of factorial (n = 4)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
```

Doubling the Argument of factorial (n = 4)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
```

Doubling the Argument of factorial (n = 4)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
```

Doubling the Argument of factorial (n = 4)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
```

Doubling the Argument of factorial (n = 4)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
```

Doubling the Argument of factorial (n = 4)

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
24
```

Doubling the Argument of factorial: Observation

Time

The number of steps “roughly” doubles. The factorial function runs in a time (linearly) proportional to the argument n.

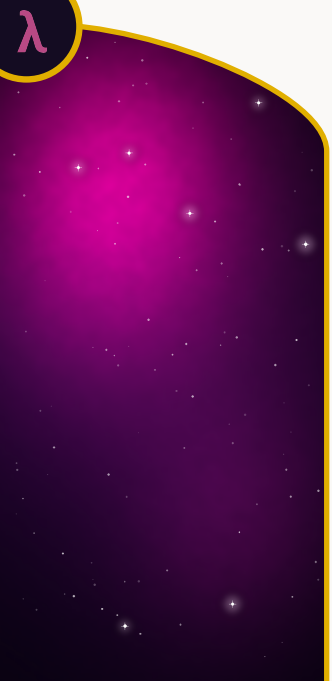
Doubling the Argument of factorial: Observation

Time

The number of steps “roughly” doubles. The factorial function runs in a time (linearly) proportional to the argument n.

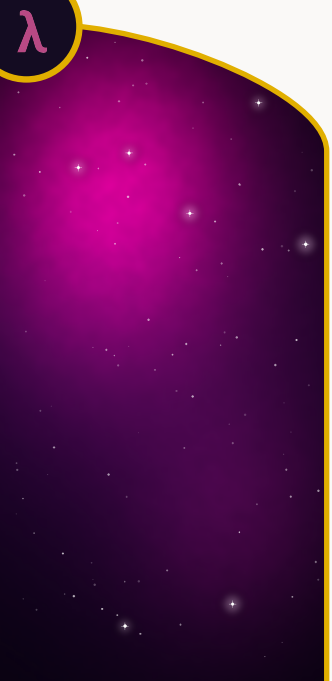
Space

The number of deferred operations also “roughly” doubles. It has a space requirement (linearly) proportional to the argument n.



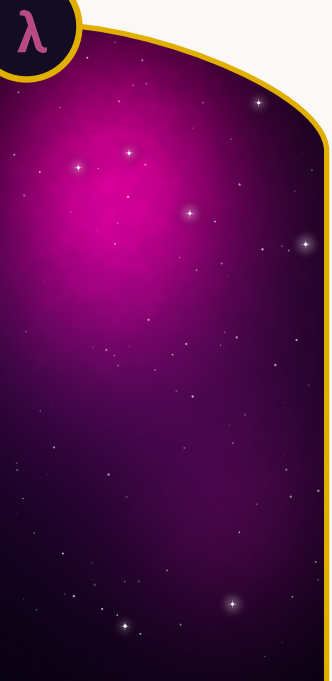
Orders of Growth

- Linear growth
  - The factorial function runs in a time (linearly) proportional to the argument n



Orders of Growth

- Linear growth
  - The factorial function runs in a time (linearly) proportional to the argument n
- Exponential growth
  - The fib function runs in a time that grows exponentially with the argument n



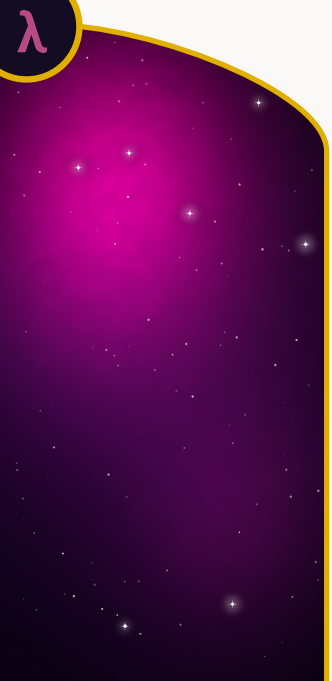
Orders of Growth

- Linear growth
  - The factorial function runs in a time (linearly) proportional to the argument n
- Exponential growth
  - The fib function runs in a time that grows exponentially with the argument n

What exactly do we mean by the above?

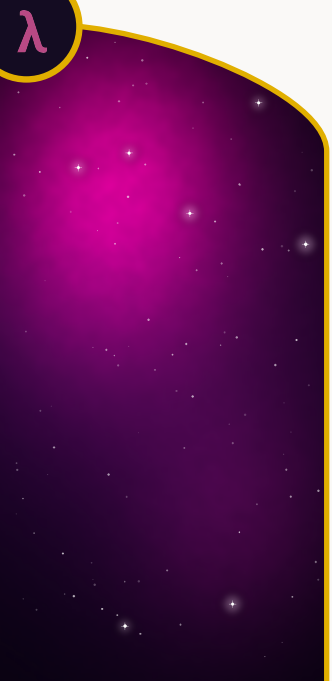
Purpose

- Rough measure
  - We are interested in a rough measure of resources used by a computational process, with respect to the problem size
- Abstraction
  - "Order of growth" is an abstraction technique
  - We decide to ignore details that we deem irrelevant
  - Examples: the processor speed of the computer, the programming environment, the programming language, or minor differences in programming style



The Resource Function

- For a program P solving a problem of size n, let $r(n)$ denote the amount of a *resource* (time or space) that P needs



The Resource Function

- For a program P solving a problem of size n, let $r(n)$ denote the amount of a *resource* (time or space) that P needs
- Recall Prof Martin and Prof Sanka: both write a program for the *same* problem

The Resource Function

- For a program P solving a problem of size n, let $r(n)$ denote the amount of a *resource* (time or space) that P needs
- Recall Prof Martin and Prof Sanka: both write a program for the *same* problem
- Plug in Martin's program: $r(n)$ tells us its resource use

The Resource Function

- For a program P solving a problem of size n, let $r(n)$ denote the amount of a *resource* (time or space) that P needs
- Recall Prof Martin and Prof Sanka: both write a program for the *same* problem
- Plug in Martin's program: $r(n)$ tells us its resource use
- Plug in Sanka's program instead: $r(n)$ now tells us Sanka's program's resource use, same symbol, different program

The Θ ("Big Theta") Notation

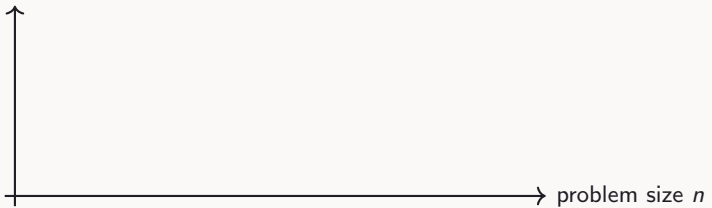
- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n

The Θ ("Big Theta") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \times g(n) \leq r(n) \leq k_2 \times g(n)$ for all $n > n_0$

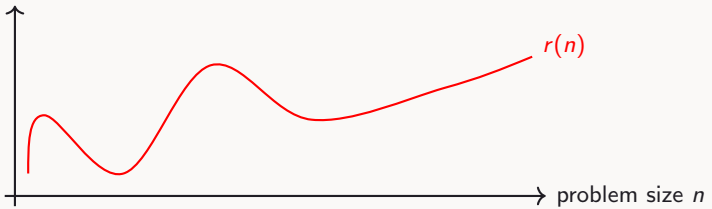
The Θ ("Big Theta") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \times g(n) \leq r\ (n) \leq k_2 \times g(n)$ for all $n > n_0$



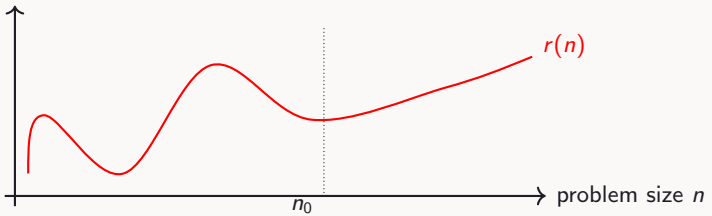
The Θ ("Big Theta") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \times g(n) \leq r(n) \leq k_2 \times g(n)$ for all $n > n_0$



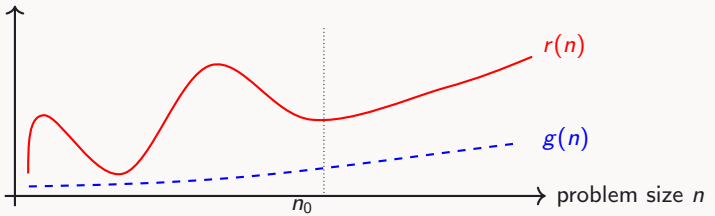
The Θ ("Big Theta") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \times g(n) \leq r(n) \leq k_2 \times g(n)$ for all $n > n_0$



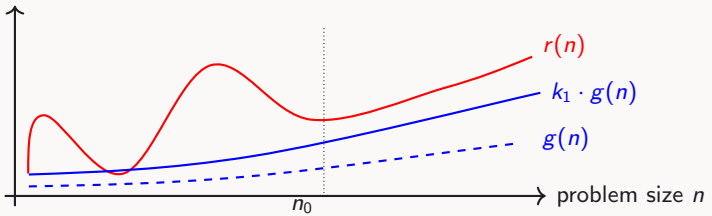
The Θ ("Big Theta") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \times g(n) \leq r(n) \leq k_2 \times g(n)$ for all $n > n_0$



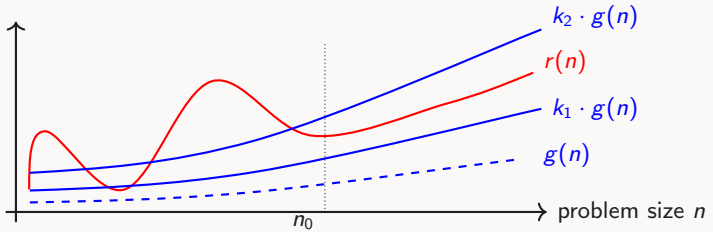
The Θ ("Big Theta") Notation

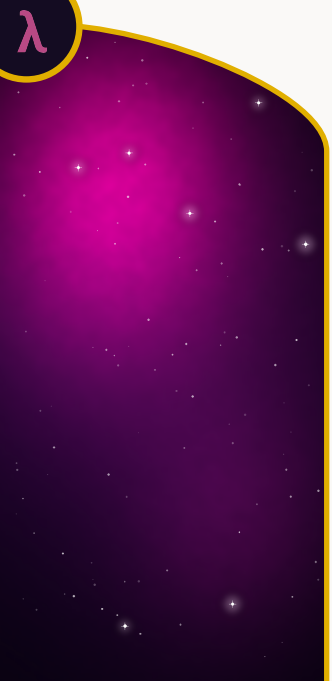
- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \times g(n) \leq r(n) \leq k_2 \times g(n)$ for all $n > n_0$



The Θ ("Big Theta") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \times g(n) \leq r(n) \leq k_2 \times g(n)$ for all $n > n_0$





The Big-O (“Big Oh”) Notation

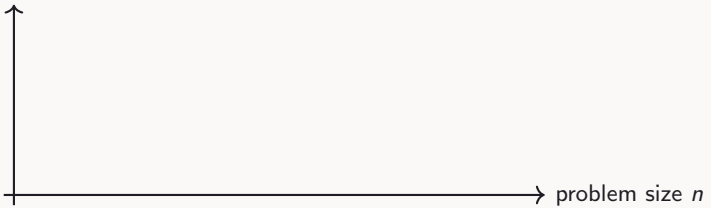
- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n

The Big-O ("Big Oh") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $O(g(n))$ if there is a positive constant k and a number n_0 such that r (n) $\leq$ k $\times$ g(n) for all n $>$ n_0

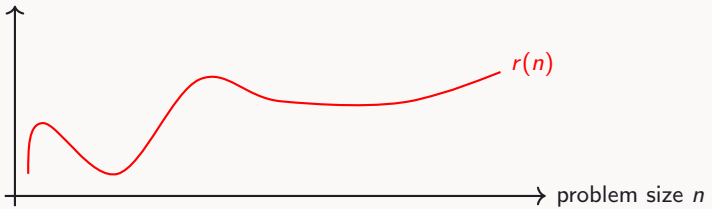
The Big-O ("Big Oh") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $O(g(n))$ if there is a positive constant k and a number n_0 such that r (n) $\leq$ k $\times$ g(n) for all n $>$ n_0



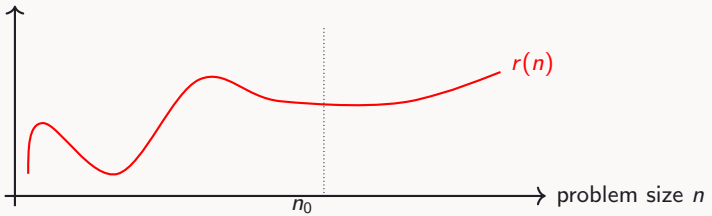
The Big-O ("Big Oh") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $O(g(n))$ if there is a positive constant k and a number n_0 such that r (n) $\leq$ k $\times$ g(n) for all n $>$ n_0



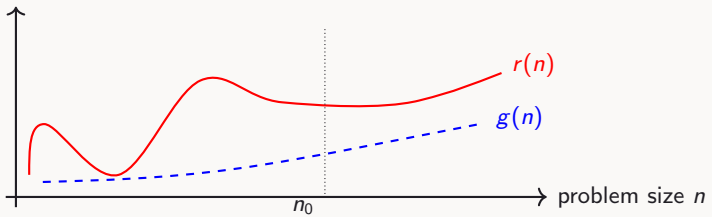
The Big-O ("Big Oh") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $O(g(n))$ if there is a positive constant k and a number n_0 such that r (n) $\leq$ k $\times$ g(n) for all n $>$ n_0



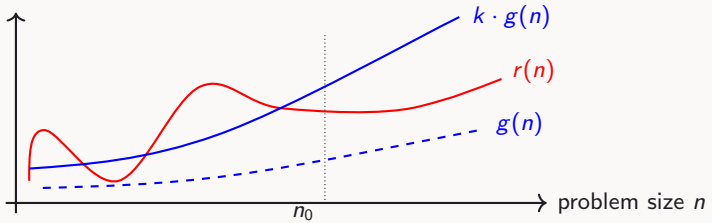
The Big-O ("Big Oh") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $O(g(n))$ if there is a positive constant k and a number n_0 such that r (n) $\leq$ k $\times$ g(n) for all n $>$ n_0



The Big-O ("Big Oh") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $O(g(n))$ if there is a positive constant k and a number n_0 such that r (n) $\leq$ k $\times$ g(n) for all n $>$ n_0



The Ω (“Big Omega”) Notation

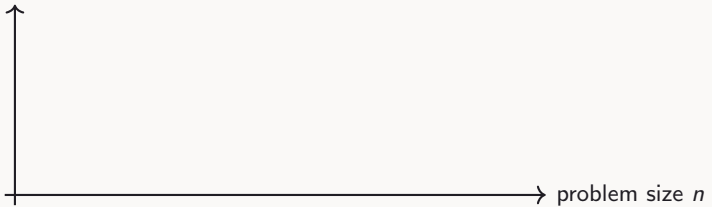
- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n

The Ω ("Big Omega") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth $\Omega(g(n))$ if there is a positive constant k and a number n_0 such that $k \times g(n) \leq r(n)$ for all $n > n_0$

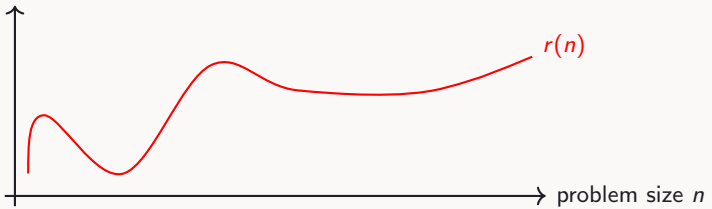
The Ω ("Big Omega") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth Ω(g(n)) if there is a positive constant k and a number n_0 such that k × g(n) ≤ r (n) for all n > n_0



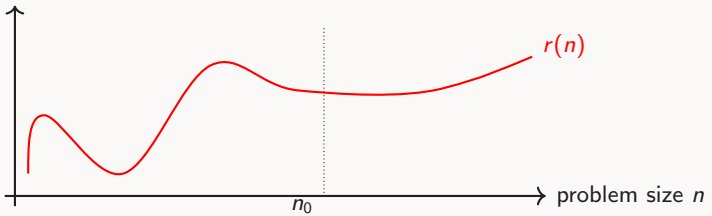
The Ω ("Big Omega") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth Ω(g(n)) if there is a positive constant k and a number n_0 such that k × g(n) ≤ r (n) for all n > n_0



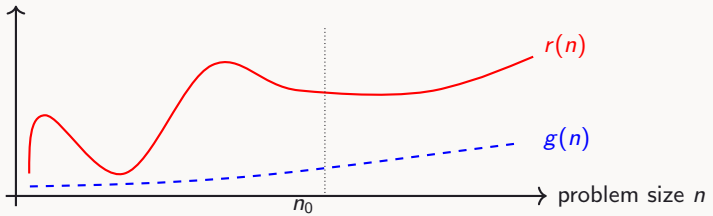
The Ω ("Big Omega") Notation

- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth Ω(g(n)) if there is a positive constant k and a number n_0 such that k × g(n) ≤ r (n) for all n > n_0



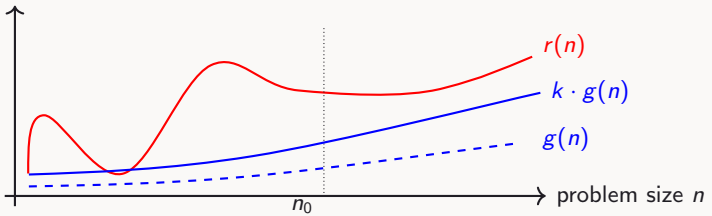
The Ω ("Big Omega") Notation

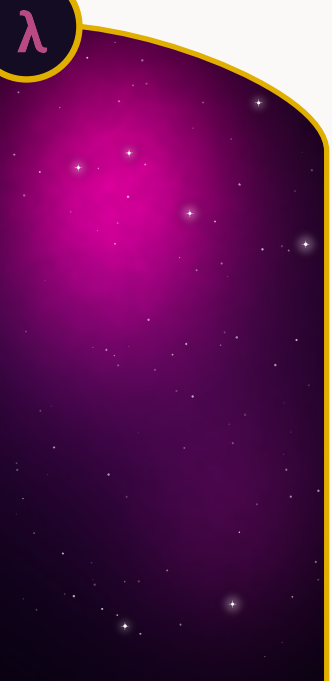
- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth Ω(g(n)) if there is a positive constant k and a number n_0 such that k × g(n) ≤ r (n) for all n > n_0



The Ω ("Big Omega") Notation

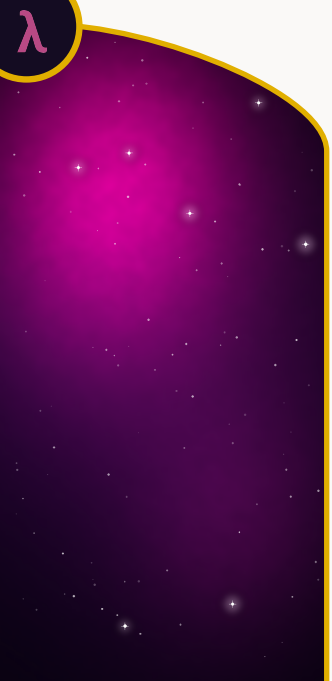
- Let n denote the size of the problem, and let r(n) denote the resource needed solving the problem of size n
- Definition: The function r has order of growth Ω(g(n)) if there is a positive constant k and a number n_0 such that k × g(n) ≤ r (n) for all n > n_0





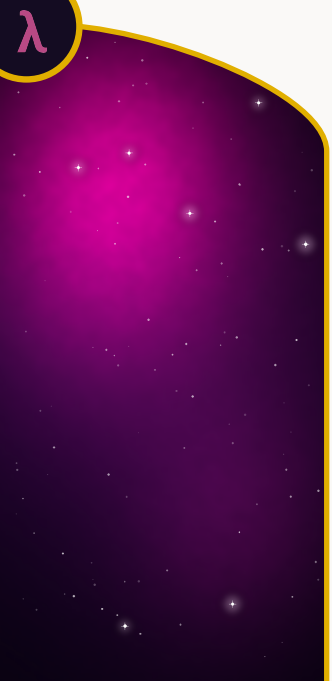
Do Constants Matter?

- Let's say r has order of growth $\Theta(n^2)$



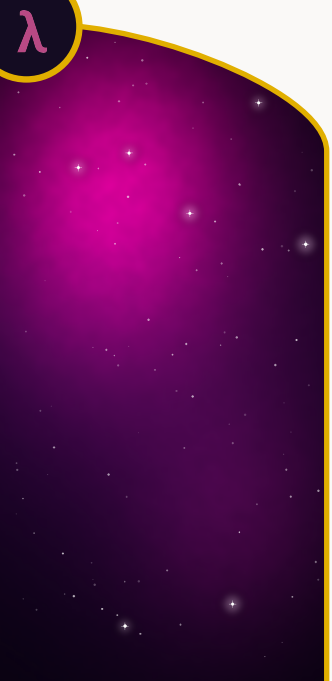
Do Constants Matter?

- Let's say r has order of growth $\Theta(n^2)$
- Does r also have order of growth $\Theta(0.5n^2)$?



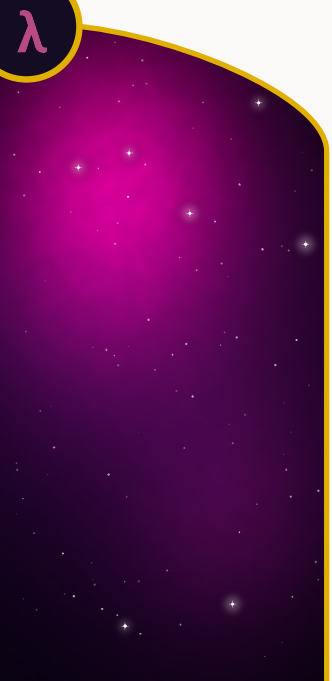
Do Constants Matter?

- Let's say r has order of growth $\Theta(n^2)$
- Does r also have order of growth $\Theta(0.5n^2)$?
- Constants don't matter
  - Because we can freely choose k, k_1, k_2



Do Minor Terms Matter?

- Let's say r has order of growth $O(n^2)$

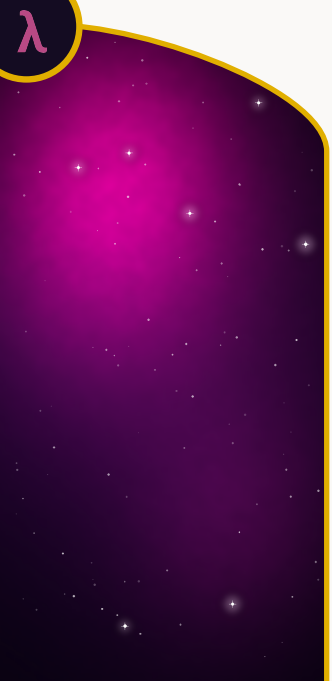


Do Minor Terms Matter?

- Let's say r has order of growth $O(n^2)$
- Does r also have order of growth $O(n^2 + 40n - 5)$?

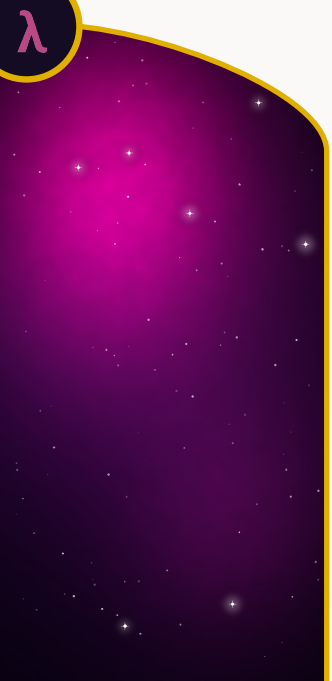
Do Minor Terms Matter?

- Let's say r has order of growth $O(n^2)$
- Does r also have order of growth $O(n^2 + 40n - 5)$?
- Minor terms don't matter
  - Because we can adjust n_0, k, k_1, k_2 such that the minor terms are overruled



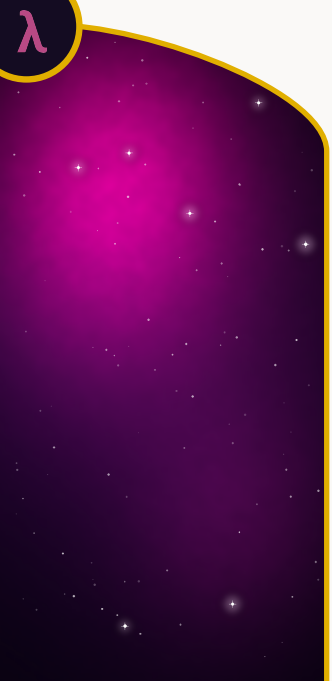
Some Common g(n)

- log n



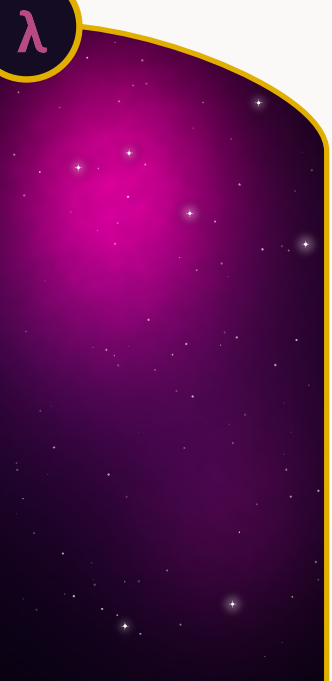
Some Common g(n)

- log n
- n



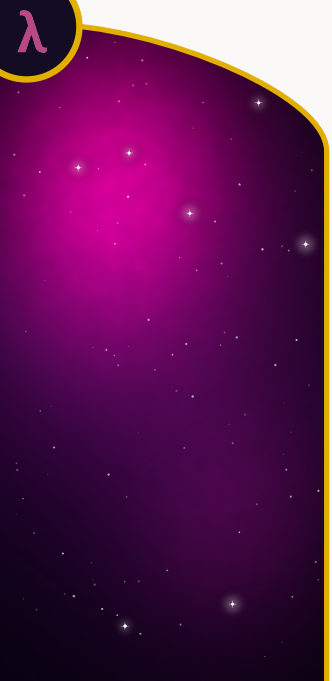
Some Common g(n)

- log n
- n
- n log n



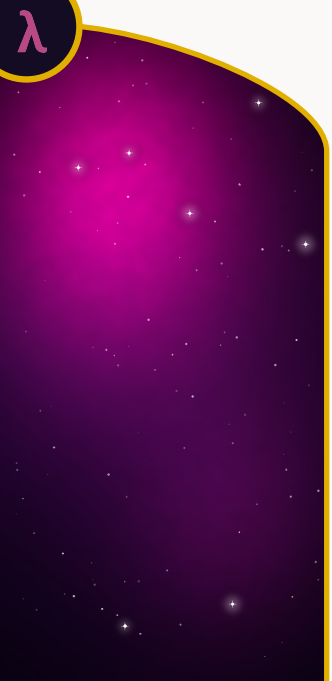
Some Common g(n)

- log n
- n
- n log n
- n^2



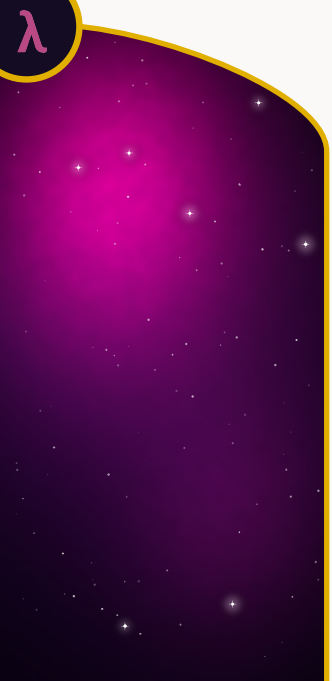
Some Common g(n)

- log n
- n
- n log n
- n^2
- n^3

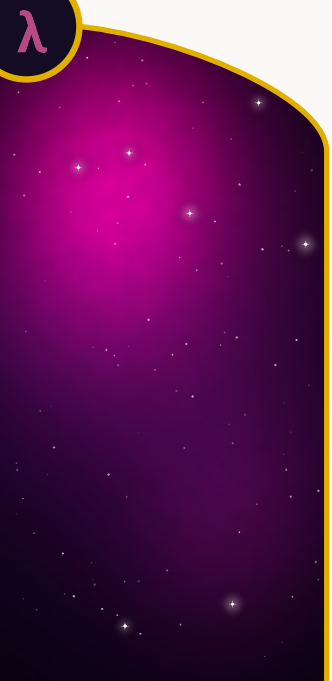


Some Common g(n)

- log n
- n
- n log n
- n^2
- n^3
- 2^n



Growing Pains



Growing Pains

It is now time (or space) to go to the Source Academy!

Apologies, I am running out of jokes here already and we are only in week 2!

Growing Pains

It is now time (or space) to go to the Source Academy!

Apologies, I am running out of jokes here already and we are only in week 2!

Look for Check-In I2B in the Source Academy!

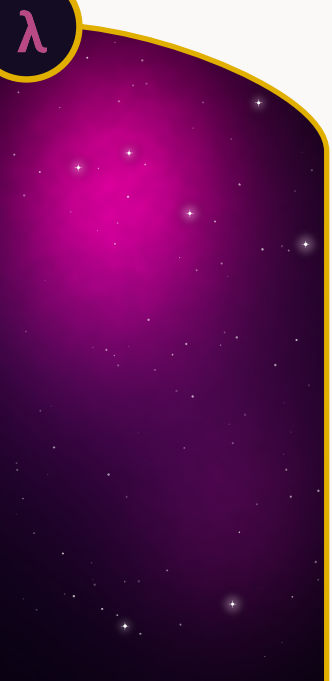
Growing Pains

It is now time (or space) to go to the Source Academy!

Apologies, I am running out of jokes here already and we are only in week 2!

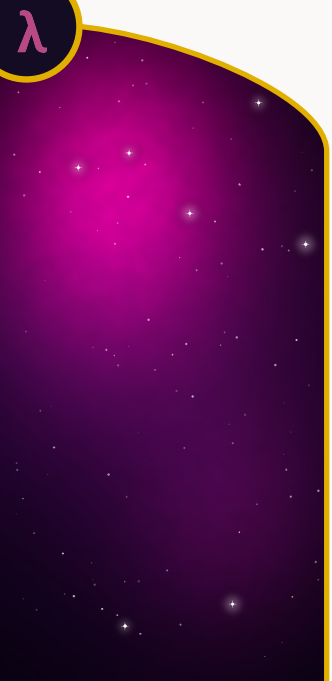
Look for Check-In I2B in the Source Academy!

(No need to finalize your Check-In submission. Please stop working as soon as we call time!)



Example Orders of Growth

- Linear time
  - The `factorial` function runs in a time that has order of growth $\Theta(n)$, where n is the function argument



Example Orders of Growth

- Linear time
  - The `factorial` function runs in a time that has order of growth $\Theta(n)$, where n is the function argument
  - Can we say its order of growth is $O(n)$?

Example Orders of Growth

- Linear time
  - The `factorial` function runs in a time that has order of growth $\Theta(n)$, where n is the function argument
  - Can we say its order of growth is $O(n)$?
  - Can we say its order of growth is $O(n^2)$?

Example Orders of Growth

- Linear time
  - The `factorial` function runs in a time that has order of growth $\Theta(n)$, where n is the function argument
  - Can we say its order of growth is $O(n)$?
  - Can we say its order of growth is $O(n^2)$?
- Exponential time
  - The `fib` function runs in a time that has order of growth $\Theta(\varphi^n)$, where n is the function argument and $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$

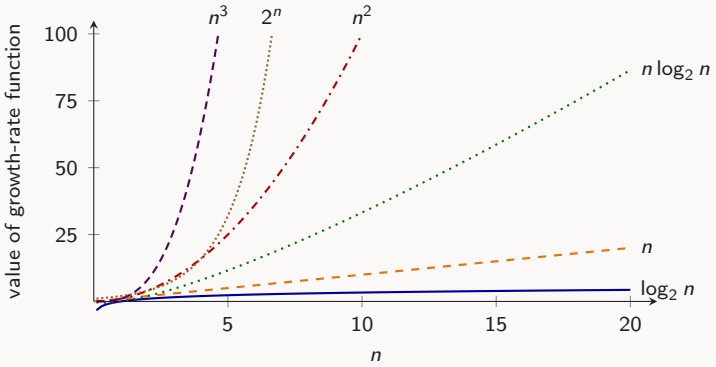
Example Orders of Growth

- Linear time
  - The `factorial` function runs in a time that has order of growth $\Theta(n)$, where n is the function argument
  - Can we say its order of growth is $O(n)$?
  - Can we say its order of growth is $O(n^2)$?
- Exponential time
  - The `fib` function runs in a time that has order of growth $\Theta(\varphi^n)$, where n is the function argument and $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$
  - Can we say its order of growth is $O(2^n)$?

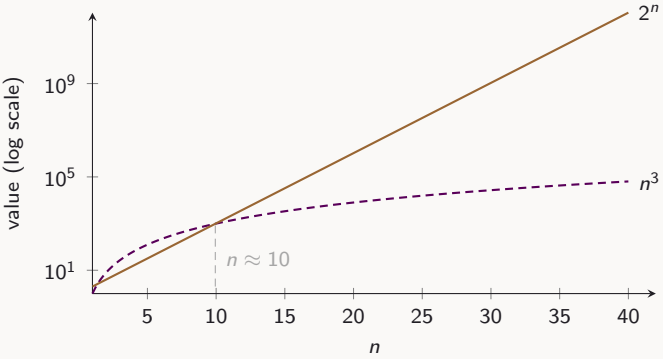
Comparing Orders of Growth: Table

	n					
Function	10	100	1,000	10,000	100,000	1,000,000
1	1	1	1	1	1	1
$\log_2 n$	3	6	9	13	16	19
n	10	10^2	10^3	10^4	10^5	10^6
$n \log_2 n$	30	664	9,965	10^5	10^6	10^7
n^2	10^2	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

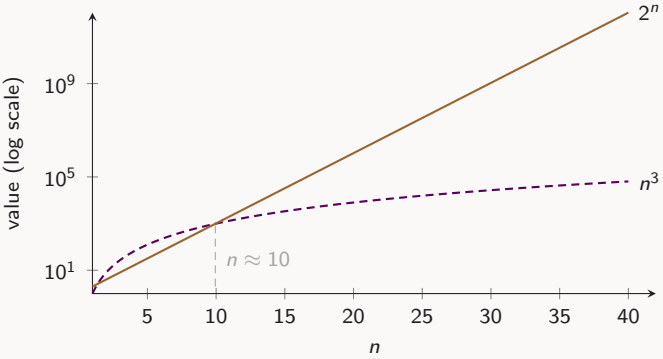
Comparing Orders of Growth: Graph



Comparing Orders of Growth: n^3 vs 2^n

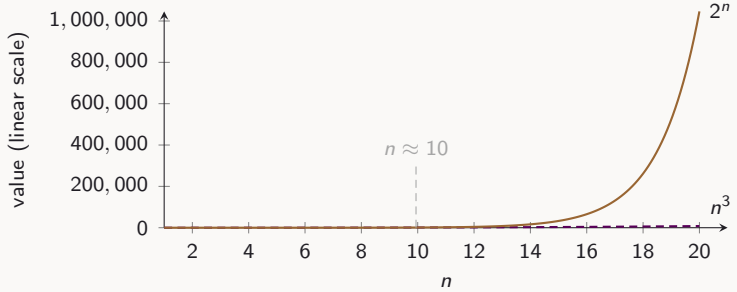


Comparing Orders of Growth: n^3 vs 2^n

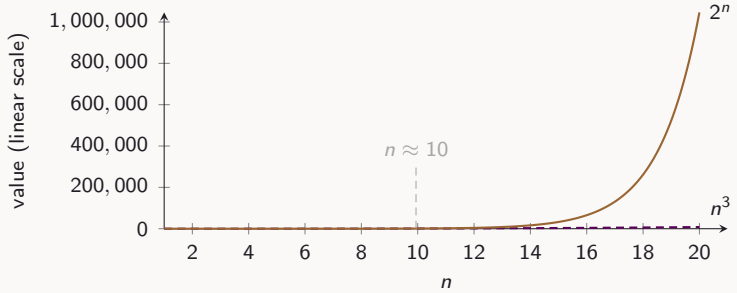


They cross around $n \approx 10$ and past that, 2^n pulls away from n^3 ever faster, and the gap grows without bound.

n^3 vs 2^n: Linear Scale



n^3 vs 2^n: Linear Scale



On a linear scale, n^3 all but disappears next to 2^n once n grows a little past the crossover this is exactly why we use order of growth instead of trying to compare raw numbers.

How to Calculate “Big Oh/Theta/Omega”

- Topic of Algorithm Analysis (CS3230)
- For us:
  - Identify the basic computational steps
  - Try a few small values
  - Extrapolate
  - Watch out for “worst case” scenarios

How to Calculate "Big Oh/Theta/Omega"

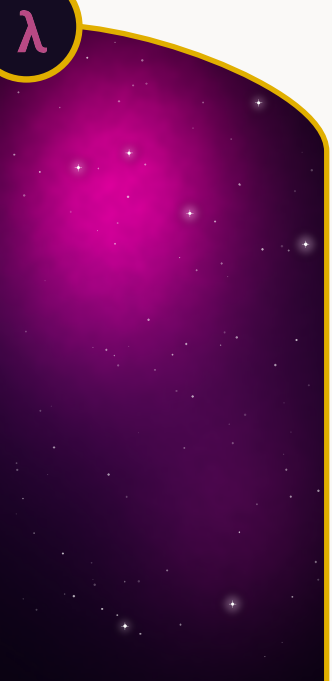
- Topic of Algorithm Analysis (CS3230)
- For us:
  - Identify the basic computational steps
  - Try a few small values
  - Extrapolate
  - Watch out for "worst case" scenarios
  - Pick the *tightest* bound

How to Calculate "Big Oh/Theta/Omega"

- Topic of Algorithm Analysis (CS3230)
- For us:
  - Identify the basic computational steps
  - Try a few small values
  - Extrapolate
  - Watch out for "worst case" scenarios
  - Pick the *tightest* bound

Why tightest?

It's true that Usain Bolt can run 100m in under 10 *minutes*... but that's not meaningful. What we actually care about is that he can do it in under 10 *seconds*.



Summary

- Resources for computational processes: time and space
- Big Theta, Big Oh, Big Omega
  - Provide abstractions or “rough” measures of resources used with respect to problem size